INGENIERÍA DE SOFTWARE I · GUÍA DE DEFENSA

Defensa del proyecto

Sistema de gestión de solicitudes de soporte técnico

Antes de la defensa

En PowerShell, dentro de la carpeta `soporte-ti`, ejecuten una vez:

```
.venv\Scripts\activate
flask --app app iniciar-bd
python -m pytest -v
```

Debe terminar con **27 passed**. Abran el sistema en vivo, el informe y el archivo `app/tickets/reglas.py`.

Presentación (3 minutos)

Qué hace. Centraliza las solicitudes de soporte: cada persona registra un ticket con código único, un coordinador lo asigna a un técnico, el técnico lo atiende y el solicitante confirma la solución. Todo queda en un historial.

Cómo está hecho. Seis componentes con una sola responsabilidad cada uno. Implementamos solo el **Módulo de Tickets**; los demás están documentados.

Cómo lo comprobamos. 18 casos de prueba (27 ejecuciones) automatizados. La primera ejecución encontró 3 defectos; los corregimos en ramas de Git y las 27 pasaron.

Resultados. 27 de 27 pruebas aprobadas, y la consulta más lenta tarda menos de 0,25 segundos con 30.000 tickets (el límite era 3).

Demostración

Usen el sistema en vivo. Contraseña de todos: `Soporte2026*` ; basta con pulsar el nombre del usuario.

1. **Ana:** registra un ticket con el asunto en blanco (muestra el error) y luego uno válido.
2. **Ana:** abre el ticket TK-000004, que es de Luis: aparece «Acceso no permitido».
3. **Carla (coordinadora):** asigna el ticket de Ana a Pedro. El técnico inactivo no aparece en la lista.
4. **Pedro:** lo pasa a En progreso e intenta resolverlo sin escribir la solución (aparece el aviso). Luego lo resuelve.
5. **Ana:** confirma y cierra el ticket. Ya no tiene acciones, pero el historial se conserva.
6. **Luis:** intenta reabrir TK-000002, resuelto hace 9 días: aparece «El plazo de 5 días terminó».

> El botón «Restablecer datos de ejemplo» devuelve el sistema a su estado inicial.

Comandos según lo que pide el docente

Lo que pide el docente	Comando	Qué decir
Casos de prueba (mínimo 12)	`python -m pytest -v`	«18 casos, 27 ejecuciones, todas en verde.»
Ver un caso concreto	`python -m pytest -v -k cp07`	«Este caso comprueba que un solicitante no puede asignar.»
Rendimiento (RNF-06)	`python scripts/medir_rendimiento.py`	«Carga 30.000 tickets y 500 usuarios; termina en CUMPLE.» Tarda unos 3 segundos.
Defectos y correcciones	`git --no-pager show 6dfb6a9`	«Esta es la validación de rol que faltaba (DEF-01).» Para salir, pulsen **q**.
Otro defecto corregido	`git --no-pager show c310d6a`	«Un solo signo, < por <=, dejaba por fuera el día 5 (DEF-03).»

Lo que pide el docente	Comando	Qué decir
Git: ramas, commits y merges	`git --no-pager log --graph --oneline --all`	«Cada funcionalidad y cada defecto tuvo su rama; todo se unió con merge.»
Respaldo (RNF-09)	`flask --app app respaldar-bd`	«Crea una copia de la base en la carpeta respaldos.»

Mostrar que las pruebas detectan errores

Cambien una sola línea, ejecuten el comando, expliquen la falla y luego restauren con `git checkout -- app`. Después de restaurar, `python -m pytest` debe volver a decir **27 passed**.

Qué cambiar	Dónde	Comando	Resultado y explicación
Poner `#` al inicio de las 3 líneas de `if not _es_gestor(usuario):`	`app/tickets/servicio.py`, líneas 113 a 115	`python -m pytest -v -k cp07`	**2 failed**, `302 == 403`: el servidor aceptó la asignación de un solicitante. Ocultar el botón no protege.
`<=` por `<` en el plazo	`app/tickets/reglas.py`, línea 86	`python -m pytest -v -k cp13`	**1 failed**: el día 5 ya no se puede reabrir. Por eso se prueban los valores límite.
`valor.strip()` por `valor`	`app/tickets/reglas.py`, línea 58	`python -m pytest -v -k cp02`	**2 failed**: un asunto con espacios llega a la base de datos y falla con error técnico.

Qué cambiar	Dónde	Comando	Resultado y explicación
`(EN_PROGRESO,)` por `(EN_PROGRESO, RESUELTO)`	`app/tickets/reglas.py`, línea 25	`python -m pytest -v -k cp09`	**1 failed**: se permite un cambio de estado que la regla RN-05 no permite.
`MAX_ASUNTO = 150` por `100`	`app/tickets/reglas.py`, línea 34	`python -m pytest -v -k cp04`	**1 failed**: un asunto de 150 caracteres, válido antes, ahora se rechaza.

Preguntas frecuentes

¿Por qué solo implementaron Tickets? Es el centro del sistema y concentra casi todos los requisitos y reglas. Los demás módulos están documentados.

¿Qué significa bajo acoplamiento aquí? Que la encuesta de satisfacción (SC-03) se podría agregar como componente nuevo sin cambiar el módulo de tickets.

¿Por qué SC-02 es perfectivo y no adaptativo? Adaptativo es cuando cambia el entorno. SC-02 es una función nueva que pide la organización.

¿Qué hizo la IA? Generó gran parte del código y las pruebas a partir de los requisitos. Nosotros revisamos cada regla, ejecutamos las pruebas, analizamos los fallos y validamos las pantallas. Respondemos por todo el código.

¿La versión en vivo es la misma del repositorio? Es una copia en el navegador con la misma interfaz y reglas, para verla sin instalar nada. La versión que se prueba con pytest es la del repositorio.